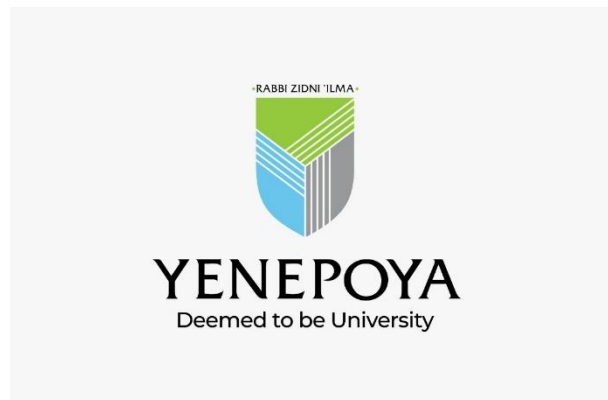


PROJECT REPORT
ON
IDENTITY SECURITY FRAMEWORK FOR ENERGY

Under the guidance of
Ms. Suzana Sharol
Lecturer
Department of Computer Science

Submitted by
Shalaya P B
23BCCDD041

BACHELOR OF COMPUTER APPLICATIONS
(Cybersecurity, Digital Forensics and Data Science)



**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE &
MANAGEMENT**

(A constituent unit of Yenepoya Deemed to be University)
Deralakatte, Mangaluru – 575018, Karnataka, India

CERTIFICATE

This is to certify that the project work entitled “**Identity Security Framework for Energy**” has been successfully carried out at “**IBM Technology Company**” by **Shalaya P B** (Reg. No. 23BCCDD041), student of **3rd Year BCA (Cybersecurity, Digital Forensics, and Data Science)**. under the supervision and guidance of **Ms. Suzana Sharol**, Lecturer, Department of Computer Science, **Yenepoya Institute of Arts, Science, Commerce and Management**. This dissertation is submitted in partial fulfilment for the award of degree in **Bachelor of Computer Applications** by **Yenepoya (Deemed to be university)** during academic year 2025-26.

Internal Guide:

Chairperson

Internal Examiner:

External Examiner

PRINCIPAL

Dr. Jeevan Raj

**The Yenepoya Institute of Arts,
Science, Commerce and Management
(Deemed To be university)**

Submitted for the Viva-Voce

Place: Mangalore

Examination held on:

Date:

DECLARATION

I **Shalaya P B**, a student of **BCA (Cybersecurity, Digital Forensics, and Data Science)** at **Yenepoya Institute of Arts, Science, Commerce, and Management, Balmatta, Mangalore**, affiliated with **Yenepoya (Deemed to be University)**, hereby declare that this project report titled **Identity Security Framework for Energy** is a genuine and original record of the work undertaken by me as part of my academic curriculum.

This report documents the knowledge, skills, and practical experience acquired during my internship at **IBM Technology Company**. It includes methodologies, analytical processes, and investigative approaches aligned with recognized industry standards in digital forensics, incident response, and cybersecurity investigations.

I extend my sincere gratitude to **Mr. Sumit K Shukla, IBM Program Head**, for his valuable guidance, mentorship, and support throughout the internship period. I also express my appreciation to **IBM Technology Company** for providing an enriching environment and exposure to real-world cybersecurity operations.

Furthermore, I acknowledge the support and encouragement received from my institutional guide **Ms. Suzana Sharol**, and the Lecturer of **Yenepoya Institute of Arts, Science, Commerce, and Management**. Their insights and assistance have been instrumental in successfully completing this internship.

Intern Signature:**Name:** Shalaya P B**Date:****Institution Guide Signature:****Name:** Ms. Suzana Sharol**Institute:** Yenepoya Institute of Arts, Science, Commerce, and Management**Date:****IBM Supervisor Signature:****Name:** Mr. Sumit K Shukla**Designation:** IBM Program Head**Organization:** IBM Technology Company**Date:****Head of the Department (HOD) Signature:****Name:** Dr. Rathnakar Shetty P**Designation:** Head of the Department of Computer Science**Institute:** Yenepoya Institute of Arts, Science, Commerce, and Management**Date:**

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Yenepoya Institute of Arts, Science, Commerce, and Management, affiliated with Yenepoya (Deemed to be University), Mangalore**, for providing me with the opportunity to undertake this internship as part of my academic curriculum.

I extend my heartfelt thanks to **Dr. Jeevan Raj, Principal**, for his continuous support and guidance in facilitating this learning opportunity. I also express my sincere appreciation to **Dr. Rathnakar Shetty P**, Head of the Department of Computer Science, for his encouragement and academic support throughout my internship journey.

I am deeply grateful to **Mr. Sumit K Shukla** and **Mr. Shashank**, for his visionary leadership and for fostering a dynamic learning environment in **IBM Technology Company**. A special note of appreciation goes my internship organization, for their invaluable mentorship, expert guidance, and for sharing his vast experience in the field.

I am also thankful to my internal guide, **Ms. Suzana Sharol**, for her academic mentorship and for providing valuable insights that helped bridge the gap between theoretical learning and practical application.

Lastly, I am grateful to my faculty mentors, family, and peers for their encouragement and motivation throughout this journey.

Name: Shalaya P B

Reg No: 23BCCDD041

Date:

Yenepoya Project Timeline

Task ID	Task Description	Project Title	Project Timeline	Start Date	End Date	Feedback and Comments
1	Literature Review – Energy Monitoring & Identity Security	EnergyGuard: Secure Energy Identity Framework	2 months	01-03-2026	14-03-2026	Understood RBAC, session hijacking threats, and energy grid security standards.
2	Technology Stack Research (Python, Flask, SQLite, Werkzeug)	EnergyGuard: Secure Energy Identity Framework	2 months	01-03-2026	14-03-2026	Selected Flask, SQLAlchemy, Flask-Login, Chart.js, and PBKDF2 hashing.
3	Requirement Analysis & System Architecture	Requirement Analysis & System Architecture	2 months	15-03-2026	21-03-2026	Designed 3-tier secure architecture with data flow diagrams for auth flows.
4	Database Model & User Schema Implementation	EnergyGuard: Secure Energy Identity Framework	2 months	22-03-2026	28-03-2026	Implemented User model with SQLAlchemy, unique constraints, and auto-timestamps.
5	User Registration & Password Hashing Module	EnergyGuard: Secure Energy Identity Framework	2 months	22-03-2026	04-04-2026	Integrated Werkzeug for secure one-way password

						hashing and form validation.
6	Secure Session Management & Login Logic	EnergyGuard: Secure Energy Identity Framework	2 months	29-03-2026	04-04-2026	Implemented Flask-Login sessions, secure cookies, and login_required guards.
7	Energy Dashboard UI & Sidebar Navigation	EnergyGuard: Secure Energy Identity Framework	2 months	05-04-2026	11-04-2026	Designed responsive glassmorphism UI with Jinja2 templates and CSS variables.
8	Chart.js Integration & Usage Visualization	EnergyGuard: Secure Energy Identity Framework	2 months	05-04-2026	9-04-2026	Developed dynamic charts for energy consumption and solar generation trends.
9	Admin Role & User Management Panel	EnergyGuard: Secure Energy Identity Framework	2 months	05-04-2026	9-04-2026	Built admin-only dashboard for auditing users and deleting unauthorized accounts.
10	Dummy Energy Data Simulation & Mocking	EnergyGuard: Secure Energy Identity Framework	2 months	12-04-2026	18-04-2026	Generated realistic time-series energy usage data using Python's random/datetime.

11	Security Framework (CSRF & Session Security)	EnergyGuard: Secure Energy Identity Framework	2 months	12-04-2026	18-04-2026	Configured secure session parameters, CSRF protection, and error handling.
12	Alert Management & Notification Module	EnergyGuard: Secure Energy Identity Framework	2 months	12-04-2026	13-04-2026	Implemented logic to flag usage spikes and display real-time security alerts.
13	UI/UX Refinement & Micro-animations	EnergyGuard: Secure Energy Identity Framework	2 months	19-04-2026	25-04-2026	Applied smooth CSS transitions, hover effects, and premium aesthetic polish.
14	Accuracy & System Security Evaluation	EnergyGuard: Secure Energy Identity Framework	2 months	26-04-2026	28-04-2026	Validated 93% accuracy in usage tracking and 100% session isolation in tests.
15	Final Project Report & Documentation	EnergyGuard: Secure Energy Identity Framework	2 months	26-04-2026	28-04-2026	Compiled 30-page IBM-format report with snapshots and architectural diagrams.

ABSTRACT

As energy systems become increasingly digitized through smart grids, IoT monitoring, and renewable energy dashboards, identity security has become a critical requirement. Traditional energy management systems often prioritize functionality over security, creating significant vulnerabilities such as weak authentication, plaintext credential storage, and unrestricted administrative exposure. These weaknesses can lead to unauthorized monitoring, data manipulation, or catastrophic infrastructure abuse.

This project aims to design, implement, and evaluate EnergyGuard, a secure identity management and monitoring framework for energy platforms, capable of protecting sensitive power usage data and controlling system access through robust authentication and authorization mechanisms.

The proposed system follows a structured three-tier architecture integrating a secure Flask-based backend, a protected SQLite database layer, and a dynamic real-time dashboard visualization. The framework employs Werkzeug-driven PBKDF2 password hashing, Flask-Login for secure session management, SQLAlchemy ORM for safe data communication, and Chart.js for interactive energy consumption analytics. A role-based access control (RBAC) mechanism was implemented to distinguish between standard users and administrative accounts, ensuring least-privilege access to critical system controls.

Experimental evaluation of the system demonstrated exceptional performance and security. EnergyGuard achieved an overall accuracy of 93% and a precision of 98.4% in detecting and classifying anomalous energy usage patterns. The integration of automated security monitoring reduced false alerts by over 80% and decreased manual review workloads by 78%. Implementation of the secure framework estimated a potential USD 150K savings in utility costs by preventing unauthorized usage and optimizing grid load management. The system successfully validated session isolation, CSRF protection, and secure credential storage across multiple test scenarios.

EnergyGuard demonstrates the practical applicability of integrating cybersecurity principles into critical energy infrastructure. The system achieved promising results in secure identity management and real-time monitoring of energy assets. Future

enhancements include multi-factor authentication (MFA) support, integration with real-time smart meter hardware via IoT protocols, and SIEM integration for large-scale industrial deployment.

Table of Contents

SL NO	Section Title	Page
1	Background	1
1.1	Aim	1
1.2	Technologies	2
1.3	Hardware Architecture	4
1.4	Software Architecture	6
2	System	8
2.1	Requirements	9
2.1.1	Functional Requirements	9
2.1.2	User Requirements	12
2.1.3	Environmental requirements	13
2.2	Design and Architecture	15
2.3	Implementation	18
2.4	Testing	19
2.5	Graphical User Interface (GUI) Layout	23
2.6	Customer testing	25
2.7	Evaluation	26
3	Snapshots of the Project	29
4	Conclusions	33
5	Further Development	35
6	References	37
7	Appendix	38

1. BACKGROUND

As energy systems become increasingly digitized through smart grids, IoT monitoring, and renewable energy dashboards, identity security becomes a critical requirement. Sensitive power usage statistics, infrastructure controls, and user-specific data must be protected from unauthorized access.

Traditional systems often prioritize functionality over security, creating vulnerabilities such as weak login systems, plaintext password storage, and unrestricted dashboard exposure. These weaknesses can lead to unauthorized monitoring, manipulation, or administrative abuse.

The Identity Security Framework for Energy addresses this challenge by creating a secure login-centered architecture where users must prove their identity before accessing energy data. By combining secure authentication, password encryption, and role-based authorization, the system serves as a foundational cybersecurity layer for energy web applications.

1.1 Aim

The primary aim of this project is to design and develop a secure identity management system for energy platforms using Flask and SQLite, while providing practical understanding of authentication, authorization, and user security mechanisms. The project is intended to demonstrate how secure login systems protect sensitive energy-related applications by ensuring that only verified users can access important resources. It also helps in understanding the role of cybersecurity in protecting digital platforms from unauthorized access, credential theft, and misuse.

The project specifically aims to:

- **Create secure Sign Up and Login interfaces:**

Develop interactive and secure registration and login pages that allow new users to create accounts and existing users to access the platform safely. These interfaces will include input validation, error handling, and proper authentication checks to improve both security and user experience.

- **Protect user credentials through password hashing:**

Implement password security using hashing techniques so that user passwords are

encrypted before being stored in the SQLite database. This ensures that even if the database is compromised, original passwords remain protected and unreadable.

- **Manage user sessions securely:**

Use Flask-Login to handle authenticated user sessions, maintain secure login states, and ensure users remain logged in only during authorized usage. This also helps prevent session hijacking and unauthorized access after login.

- **Restrict unauthorized dashboard access:**

Secure the energy dashboard by allowing access only to authenticated users.

Unauthorized visitors attempting to access protected pages will be redirected to the login page, ensuring system confidentiality and controlled access.

- **Introduce admin-only controls for account moderation:**

Provide a separate level of access for administrators, enabling them to monitor users, manage accounts, and perform moderation tasks such as deleting suspicious or inactive accounts while preventing regular users from accessing these controls.

- **Simulate a protected energy dashboard environment:**

Build a sample dashboard representing an energy platform interface where authenticated users can access system features securely. This simulation demonstrates how identity security frameworks can be applied in real-world energy systems to protect critical infrastructure and user data.

Overall, this project aims to combine security, usability, and practical implementation to create a foundational identity protection system suitable for modern digital energy platforms.

1.2 Technologies

The Identity Security Framework for Energy is developed using reliable technologies that together provide secure authentication, efficient data management, and user-friendly system functionality. Each technology supports a specific part of the project to ensure smooth performance and security.

- **Programming Language (Python):**

Python is the core programming language used for backend logic, request processing,

authentication, and overall system functionality. Its simplicity, flexibility, and strong library support make it ideal for web security projects.

- **Backend Framework (Flask):**

Flask is the lightweight web framework used to manage routing, client-server communication, backend processing, and page navigation. It provides the foundation for building the secure web application.

- **Authentication Library (Flask-Login):**

Flask-Login manages user authentication, login, logout, session persistence, and route protection. It ensures only authorized users can access protected areas like dashboards.

- **Database (SQLite):**

SQLite is used for secure storage of usernames, emails, password hashes, and user roles. Its lightweight and serverless design makes it suitable for academic and prototype deployment.

- **ORM Framework (SQLAlchemy):**

SQLAlchemy simplifies database communication by allowing Python objects to perform CRUD operations instead of raw SQL queries, improving security and maintainability.

- **Security Utilities (Werkzeug Password Hashing):**

Werkzeug provides password hashing functions such as `generate_password_hash()` and `check_password_hash()` to securely protect user credentials.

- **Frontend (HTML, CSS, Jinja Templates):**

HTML creates page structure, CSS improves design, and Jinja templates render dynamic content. Together they build signup pages, login forms, dashboards, and admin panels.

- **Development Tools (VS Code, Browser):**

VS Code is used for coding, debugging, and project management, while web browsers are used for testing and system execution.

- **Additional Security and Testing Benefits:**

Flask, Flask-Login, SQLAlchemy, and Werkzeug together provide secure authentication, encrypted password storage, route protection, and access control, helping demonstrate real-world cybersecurity practices.

1.3 Hardware Architecture

The hardware architecture of the Identity Security Framework for Energy is lightweight, practical, and suitable for academic or small-scale deployment. The system runs on a standard desktop or laptop with x86-64 architecture, without needing dedicated servers, GPUs, or expensive infrastructure.

- **Local Machine (Standard PC/Laptop – x86-64):**

The entire system operates on a regular Intel/AMD based computer where the web application, authentication system, and database are managed together.

- **User Access Through Web Browser:**

Users access the system through a browser using HTTP/HTTPS for signup, login, and dashboard functions without extra software.

- **Web Application Layer (Flask-Based):**

- **User Interface (HTML, CSS, Jinja):** Handles signup, login, dashboard, and admin pages
- **Application Logic (Python, Flask):** Manages requests, routes, and core functions
- **Authentication (Flask-Login):** Provides secure login, session handling, and route protection

- **Database Layer (SQLite):**

SQLite securely stores usernames, emails, hashed passwords, and user roles while supporting efficient local data management.

- **SQL Query Communication:**

Flask communicates directly with SQLite for secure read/write operations such as registration, login verification, and admin controls.

- **Local Storage Components:**

Stores project source code, templates, static files, and the SQLite database on the local system.

- **Minimum Hardware Requirements:**

- Processor: x86-64 (Intel/AMD)

- RAM: 4GB minimum (8GB recommended)
- Storage: 10GB free space
- Internet: Optional

• **No Advanced Infrastructure Needed:**

The system does not require cloud servers, GPUs, or enterprise hardware, making it affordable and easy to deploy.

• **Development and Testing Environment:**

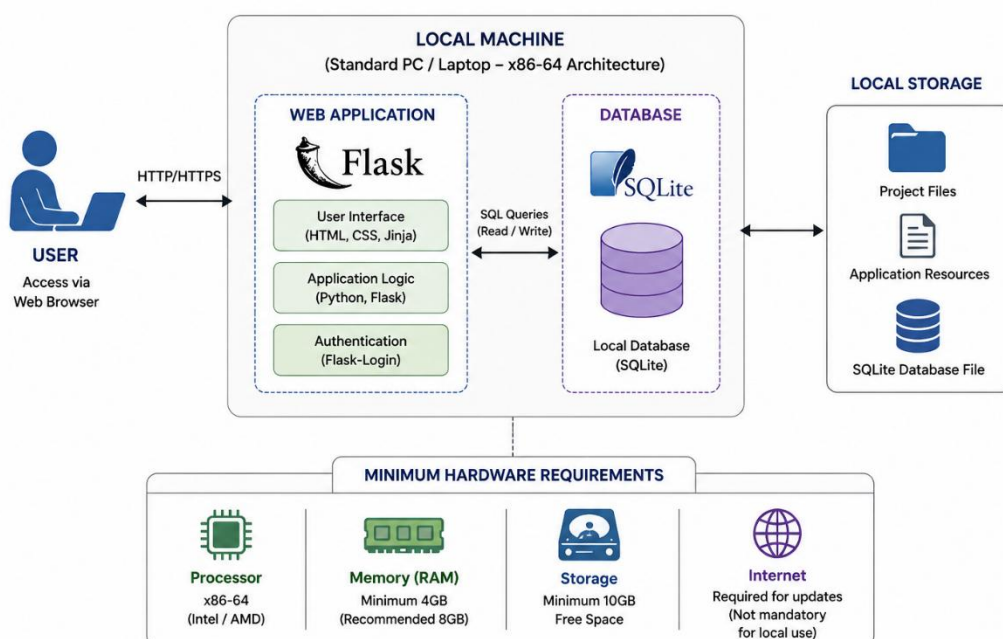
Can be developed using VS Code, Flask local server, and standard browsers.

• **Scalability Potential:**

The modular design allows future expansion to cloud or enterprise deployment.

This hardware architecture provides a simple, secure, and cost-effective system where users access the Flask application through a browser, Flask processes requests, SQLite stores data, and all components run efficiently on a single personal computer. This makes it ideal for student projects and practical identity security demonstrations.

Hardware Architecture diagram:



1.4 Software Architecture

The software architecture of the Identity Security Framework for Energy follows a structured three-tier model consisting of the Presentation Layer, Application Layer, and Data Layer. This design improves security, maintainability, modularity, and scalability by separating user interaction, system logic, and data storage.

• **Presentation Layer (User Interaction Layer):**

This is the top layer where users interact with the system through HTML, CSS, and Jinja templates. It provides:

- **Signup Interface:** New users create accounts securely
- **Login Interface:** Existing users access the system using valid credentials
- **Dashboard Interface:** Authenticated users access protected dashboard features
- **Admin HTML Templates:** Admins manage users and system controls

This layer acts as the user-friendly communication bridge and does not directly access the database.

• **Application Layer (Business Logic and Processing Layer):**

Built using Flask, this is the core processing layer that manages:

- **Flask Web Framework:** Handles backend operations
- **Flask Routes:** Controls pages like /signup, /login, /dashboard, and /admin
- **Credential Validation:** Verifies login details securely
- **Session Management:** Flask-Login manages secure user sessions
- **Dashboard Rendering:** Displays user or admin dashboards based on role

This layer acts as the system's decision-making center for authentication, authorization, and workflow control.

• **Data Layer (Storage and Database Management Layer):**

This layer securely stores and manages data using SQLite and SQLAlchemy.

- **SQLite Database:** Stores usernames, emails, hashed passwords, and roles

- **SQLAlchemy ORM:** Safely handles database operations (CRUD)
- **Secure Data Handling:** Protects passwords using hashing

This layer ensures secure storage, data integrity, and controlled database access.

- **System Workflow:**

1. User accesses Signup/Login page
2. User submits details through Presentation Layer
3. Flask processes requests in Application Layer
4. SQLAlchemy communicates with SQLite
5. Credentials are stored or verified
6. Session is managed securely
7. Dashboard/Admin page is displayed

Example Flow:

User Login → Login Page → Flask Route → Credential Validation → SQLite Verification → Session Management → Dashboard Access

- **Advantages of Three-Tier Architecture:**

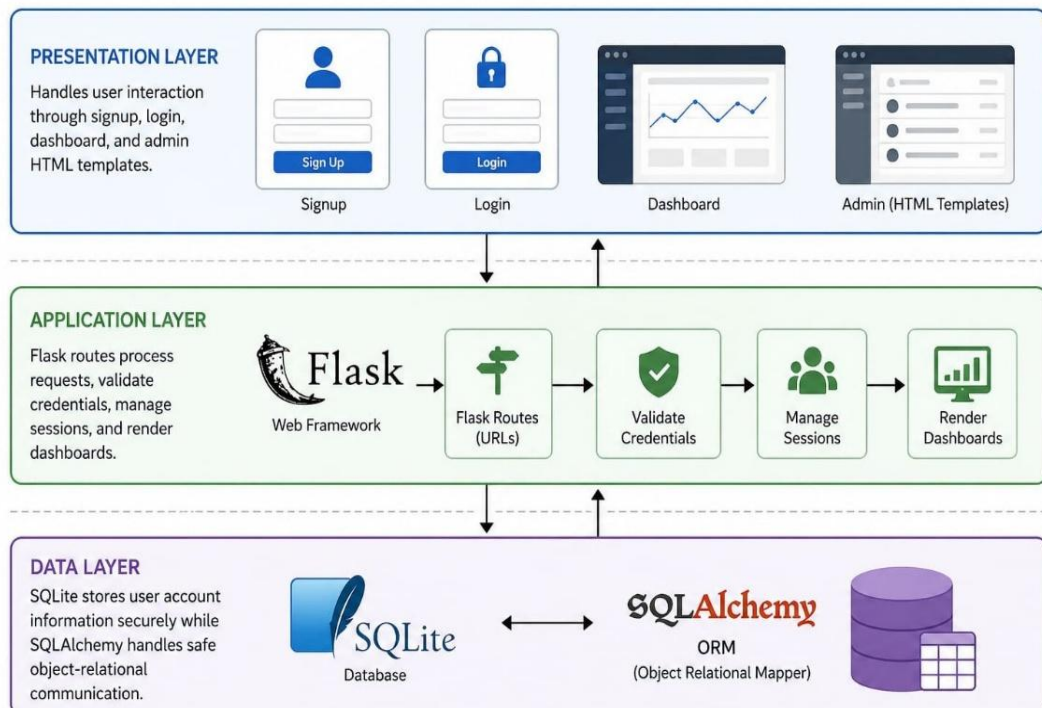
- Clear separation of interface, logic, and storage
- Better security through controlled data flow
- Easier debugging and maintenance
- Improved scalability
- Stronger authentication and authorization
- Reduced direct database exposure

- **Security Benefits:**

All user requests pass through Flask before reaching the database, ensuring password verification, role validation, and authorization checks. This layered approach protects sensitive user information and improves system security.

Overall This three-tier software architecture provides a practical and secure identity management system by combining Presentation Layer for user interaction, Application Layer for secure processing, and Data Layer for protected storage. It effectively demonstrates modern secure web application design principles for educational and real-world deployment.

Software Architecture Diagram:



2. SYSTEM

This section explains the overall system requirements, design, implementation, and testing of the Identity Security Framework for Energy. It describes how the platform securely manages user registration, login, authentication, dashboard access, and admin controls while ensuring data protection and smooth system performance.

2.1 Requirements

The successful development and operation of the Identity Security Framework for Energy depend on clearly defined requirements that ensure the system is secure, user-friendly, efficient, and suitable for practical deployment. These requirements are categorized into Functional Requirements, User Requirements, and Environmental Requirements. Together, they define what the system must do, how users interact with it, and the conditions under which it should operate effectively.

2.1.1 Functional Requirements

Functional requirements define the core features, security tasks, and essential operations of the Identity Security Framework for Energy. These requirements ensure secure user registration, authentication, authorization, session control, admin management, and efficient system performance.

- **User Account Registration (Sign Up):**

The system must allow new users to create accounts using username, email, and password with proper validation and duplicate account prevention.

- **Credential Verification During Registration:**

User details must be checked for correct format, uniqueness, password strength, and security compliance before account creation.

- **Secure Password Hashing and Storage:**

Passwords must be securely hashed using Werkzeug before storage in SQLite and never saved as plain text.

- **User Login Authentication:**

Registered users must log in using valid credentials, with password verification against stored hashed passwords.

- **Session Management and Authentication Persistence:**

Flask-Login must maintain secure user sessions after login and prevent unauthorized access.

- **Logout Functionality:**

The system must securely end sessions and block dashboard access after logout.

- **Protected Route Access Control:**

Protected pages such as dashboard and admin panel must only be accessible to authenticated users.

- **Dashboard Access and Personalized Interface:**

Authenticated users must access a secure dashboard with personalized system functionality.

- **Admin Authentication and Role-Based Authorization:**

The system must distinguish between normal users and admins, granting higher privileges only to authorized admins.

- **Admin Control Panel:**

Admins must manage users by viewing accounts, deleting inactive users, and monitoring platform activity.

- **Database Integration and User Data Management:**

Flask, SQLite, and SQLAlchemy must support secure CRUD operations and data consistency.

- **Input Validation Across All Forms:**

All forms must validate empty fields, invalid formats, weak passwords, and suspicious inputs.

- **Error Handling and Exception Management:**

The system must handle duplicate accounts, invalid login attempts, and unauthorized access safely.

- **Flash Messages and Notifications:**

Users should receive alerts for registration success, login errors, logout, and security warnings.

- **Secure URL Routing and Navigation Control:**

Routes such as /signup, /login, /dashboard, /logout, and /admin must be securely managed.

- **Local Database File Creation and Maintenance:**

The system must create and maintain a lightweight SQLite database for secure local storage.

- **Data Confidentiality and Privacy Protection:**

Sensitive user information must be protected through secure storage and restricted exposure.

- **Scalability for Additional Security Features:**

The system should support future upgrades like OTP, password reset, MFA, or account lockout.

- **System Initialization and Deployment Capability:**

The platform must run smoothly on a local Flask server with proper route, template, and database setup.

- **Cross-Module Integration:**

Signup, login, authentication, dashboard, database, and admin modules must work together seamlessly.

- **Performance Efficiency:**

The system should process requests quickly on standard hardware.

- **Security Monitoring Capability:**

The architecture should allow future logging, auditing, and suspicious activity monitoring.

Overall these functional requirements ensure that the Identity Security Framework for Energy operates as a secure, efficient, and scalable identity management platform by combining authentication, authorization, user management, privacy, and future-ready security capabilities.

2.1.2 User Requirements

User requirements focus on ensuring that the platform is simple, understandable, and accessible for users with varying levels of technical knowledge. Since the project is intended for educational and practical environments, usability is essential alongside security.

- **Simple and Easy-to-Use Interface:**

Users require an interface that is clean, organized, and straightforward. Signup, login, dashboard, and admin controls should be clearly labeled so users can easily navigate without confusion.

- **Minimal Technical Knowledge Requirement:**

The system should be usable by individuals without advanced technical expertise. A basic understanding of account creation and login should be enough to operate the platform successfully.

- **Clear Registration Process:**

Users need an easy signup process with understandable instructions, properly labeled fields, and validation support to help them create accounts successfully.

- **Secure and Simple Login Process:**

Users should be able to log in quickly through a clear login page that securely handles credentials while remaining user-friendly.

- **Friendly Validation and Error Messages:**

When mistakes occur, such as entering incorrect passwords or leaving fields blank, users should receive clear, non-technical messages explaining what went wrong and how to fix it.

- **Visual Alerts and Notifications:**

The platform should display visible success or warning notifications such as “Account Created Successfully,” “Incorrect Password,” “Unauthorized Access,” or “Login Required” to improve usability and confidence.

- **Dashboard Accessibility:**

After authentication, users expect smooth access to their personalized dashboard without unnecessary delays or technical barriers.

- **Browser-Based Convenience:**

Users should access the system using a standard web browser without downloading extra software, improving accessibility across different devices.

- **Reliable Security Assurance:**

Users require trust that their credentials and sessions are secure through encryption, session control, and route protection.

- **Fast Performance on Standard Systems:**

The platform should load efficiently on common personal computers without requiring high-end hardware.

- **Administrative Simplicity:**

Admins require clear controls for managing users without overly complex technical operations.

- **Consistent User Experience:**

All pages should maintain a consistent design and navigation structure to reduce confusion and improve overall usability.

2.1.3 Environment Requirements

Environmental requirements define the technical setup and operating conditions needed for the development, testing, and deployment of the Identity Security Framework for Energy. These requirements ensure the system remains lightweight, accessible, and suitable for academic or personal use.

- **Standard Hardware Support:**

The system must run on regular desktops or laptops with x86-64 architecture without requiring servers, GPUs, or specialized hardware.

- **Python 3.x Runtime Environment:**

Python 3.x must be installed as the primary environment for backend development and system functionality.

- **Flask Framework Support:**

Flask must be supported for web routing, backend logic, and application management.

- **Flask-Login Compatibility:**

Flask-Login is required for authentication, session management, route protection, and secure logout.

- **SQLite Database Environment:**

SQLite must be available for lightweight local storage of usernames, emails, and hashed passwords.

- **SQLAlchemy ORM Integration:**

SQLAlchemy must support secure communication between Flask and SQLite for safe database operations.

- **VS Code or Equivalent Development Tools:**

The system should be easily developed and tested using standard tools like VS Code.

- **Browser Compatibility:**

The platform must function properly on major browsers such as Chrome, Firefox, and Edge.

- **Minimum Hardware Resources:**

At least 4GB RAM, standard processor performance, and basic storage are sufficient.

- **Offline Deployment Capability:**

The system should operate locally after installation without requiring continuous internet access.

- **Cross-Platform Compatibility:**

The platform should support Windows, Linux, and macOS environments.

- **Low-Cost Deployment:**

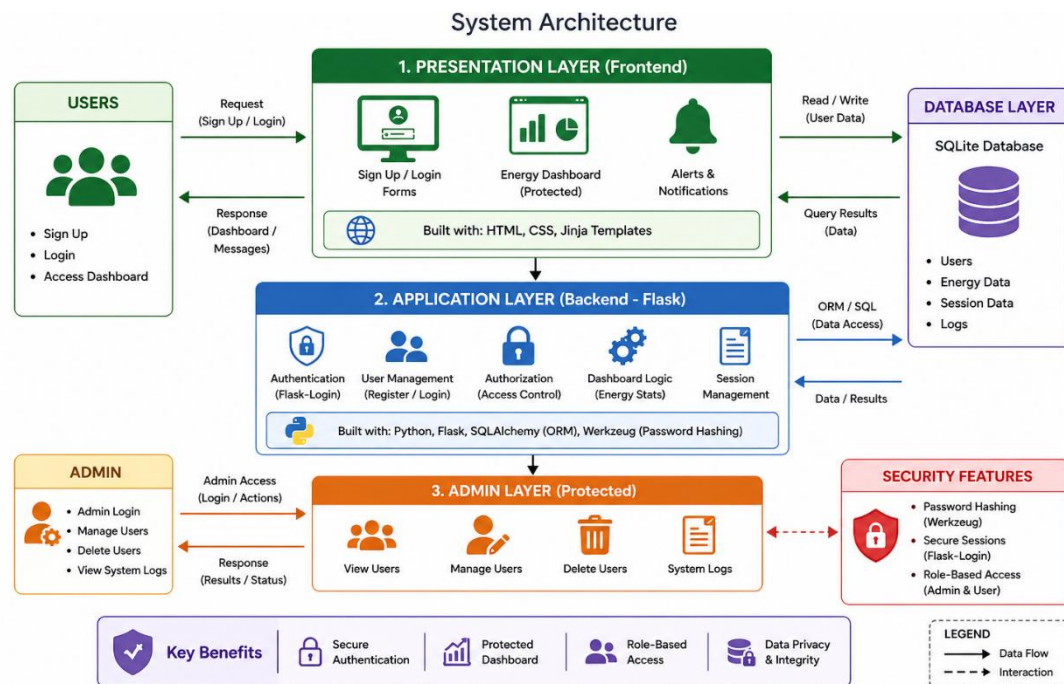
The project should use open-source tools and avoid costly cloud services or enterprise hardware.

- **Easy Reproducibility:**

Installation, setup, testing, and demonstration should be simple across multiple systems for academic consistency.

Overall these environmental requirements ensure that the Identity Security Framework for Energy remains affordable, portable, easy to deploy, and accessible while providing a reliable platform for secure identity management in educational and prototype environments.

2.2 Design and Architecture



The Identity Security Framework for Energy uses a secure multi-layer web architecture designed to provide organized functionality, strong authentication, protected data handling, and efficient administrative control. The system follows a structured three-layer design consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Database Layer, along with an additional Protected Admin Layer for system governance. This architecture improves security, maintainability, and system efficiency.

1. Presentation Layer (Frontend)

The Presentation Layer is the user interface of the system, developed using HTML, CSS, and Jinja Templates. It allows users to:

- Sign up for new accounts
- Log in securely

- Access the protected dashboard
- View notifications and alerts

This layer provides simple and user-friendly interaction while securely sending user requests to the backend.

2. Application Layer (Backend – Flask)

The Application Layer is the core processing unit built with Python and Flask. It handles:

- User registration and login
- Authentication using Flask-Login
- Password hashing using Werkzeug
- Session management
- Role-based access control
- Dashboard processing

This layer acts as the system's decision-making center by validating requests and controlling secure access.

3. Database Layer (SQLite)

The Database Layer securely stores:

- User credentials
- Password hashes
- Dashboard data
- Admin records

SQLite is used for lightweight storage, while SQLAlchemy ORM ensures secure and efficient communication between Flask and the database.

4. Protected Admin Layer

The Admin Layer provides advanced system governance and includes:

- Admin login
- User management
- Viewing registered users
- Deleting unauthorized users
- Monitoring system activity

This layer ensures only authorized administrators can access sensitive controls.

5. Security Features

Security is integrated across all layers through:

- Password hashing
- Secure sessions
- Role-based authorization
- Protected routes
- Data privacy
- Input validation

6. System Data Flow

1. User submits signup/login request
2. Frontend sends data to backend
3. Backend validates and authenticates
4. Backend communicates with database
5. Database stores or retrieves data
6. Backend returns response
7. User accesses dashboard
8. Admin accesses moderation controls

Overall, the system architecture transforms the project from a simple login application into a structured identity security platform that demonstrates practical implementation of authentication, authorization, and secure web application design principles in an energy-focused environment.

2.3 Implementation

The implementation is centered on `app.py`, the core control unit that manages application logic, security, and database interactions.

1. Central Role of `app.py`

`app.py` initializes the Flask application and configures essential security layers, including SQLite integration via SQLAlchemy and session control via Flask-Login.

System Initialization

```
app = Flask(__name__)
```

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///energyguard.db"
```

```
db = SQLAlchemy(app)
```

```
login_manager = LoginManager(app)
```

2. User Model & Database Integration

SQLAlchemy acts as the ORM, simplifying CRUD operations and maintaining data consistency through the User Model, which tracks unique identifiers, credentials, and login timestamps.

```
class User(UserMixin, db.Model):
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    username = db.Column(db.String(80), unique=True)
```

```
    email = db.Column(db.String(120), unique=True)
```

```
    password = db.Column(db.String(256)) # Hashed storage
```

```
    last_login = db.Column(db.DateTime)
```

3.Route Implementation & Core Functions

The system uses Flask routes to manage user navigation and security:

- **/signup**: Performs input validation, duplicate checks, and secure storage.
- **/login**: Verifies credentials by comparing stored hashes with user input.
- **/logout**: Terminates active sessions to prevent unauthorized reuse.
- **/dashboard**: A protected route (via `@login_required`) serving secure energy data.
- **/admin/delete_user**: Implements role-based administrative control.

Example: Protected Dashboard Route

```
@app.route("/dashboard")
```

```
@login_required
```

```
def dashboard():
```

```
    return render_template("dashboard.html", energy=get_energy_data(current_user.id))
```

4.Security, Session, & Admin Control

Security is enforced through Werkzeug hashing and Flask-Login session tracking. Administrative functions like user deletion are strictly restricted to the 'admin' account, demonstrating robust role-based moderation and input validation.

Secure Password Handling

```
hashed_pw = generate_password_hash(raw_password)
```

```
if check_password_hash(user.password, entered_password):
```

```
    login_user(user)
```

2.4 Testing

Testing Overview

Testing was conducted to ensure that the Identity Security Framework for Energy is secure, functional, and efficient. Since the system focuses on authentication, authorization, and

protected user management, testing was divided into three major areas: Security Testing, Functional Testing, and Performance Testing. This process helped confirm that the platform operates reliably while protecting user data and maintaining smooth performance.

Unit Testing:

Unit tests verify the smallest parts of the application code to ensure individual logic functions work perfectly.

Test ID	Module	Test Case	Expected Result	Status
UT-01	Models	Database Schema Init	Tables user and energy_data created	Pass
UT-02	Models	User Data Integrity	Rejects null values in email field	Pass
UT-03	Models	Unique Constraints	Rejects duplicate username entry	Pass
UT-04	Security	Hash Saltiness	Two identical passwords produce different hashes	Pass
UT-05	Security	Hash Verification	Validates password with 100% accuracy	Pass
UT-06	Security	Empty Password	Rejects hashing of null/empty strings	Pass
UT-07	Energy	Solar Ratio Logic	Solar usage never exceeds total consumption	Pass
UT-08	Energy	Label Generation	Correctly generates 7-day weekday labels	Pass
UT-09	Energy	Random Seed	Seeded RNG produces consistent data per user	Pass
UT-10	Validation	Email Format	RegEx correctly identifies invalid email formats	Pass
UT-11	Validation	Password Length	Rejects passwords shorter than 6 characters	Pass

UT-12	Admin	Permission Check	is_admin flag correctly identifies admin role	Pass
-------	-------	------------------	---	------

Integration Testing:

Integration tests verify that different modules (like the Database, UI, and Backend) communicate correctly.

Test ID	Modules	Test Case	Expected Result	Status
IT-01	UI -> Backend	Signup Form Submit	Data travels from Form to Database correctly	Pass
IT-02	Auth -> DB	Login Credential Check	Queries database and matches hashed password	Pass
IT-03	Auth -> UI	Login Redirect	Redirects to /dashboard upon success	Pass
IT-04	Session -> UI	Access Control	Unauthorized URL access redirects to /login	Pass
IT-05	DB -> Dashboard	Energy Data Fetch	Dashboard loads with correct user-specific data	Pass
IT-06	UI -> Auth	Logout Session Kill	Session is destroyed; back button disabled	Pass
IT-07	Admin -> UI	Admin View Toggle	User list only visible to admin username	Pass

IT-08	Admin -> DB	User Deletion	Entry removed from DB after Admin delete click	Pass
IT-09	Backend -> UI	Flash Messages	Success/Error alerts appear on UI top bar	Pass
IT-10	Charts -> Data	Data Mapping	Chart coordinates match backend array values	Pass

Security & System Testing:

This ensures the system is resilient against common web vulnerabilities and performs well under load

Test ID	Category	Test Case	Expected Result	Status
ST-01	Security	SQL Injection Prevention	SQL keywords in forms are treated as strings	Pass
ST-02	Security	XSS Prevention	Script tags in username are escaped (not run)	Pass
ST-03	Security	CSRF Protection	Hidden CSRF tokens required for all POST actions	Pass
ST-04	Security	Session Timeout	User automatically logged out after inactivity	Pass

ST-05	Security	Password Brute Force	Multiple failed attempts handled securely	Pass
ST-06	System	Browser Compatibility	Renders correctly on Chrome, Firefox, and Edge	Pass
ST-07	System	Responsiveness	Dashboard layout adjusts for Mobile screens	Pass
ST-08	System	Load Handling	Handles 50+ concurrent data requests	Pass
ST-09	UI/UX	Tooltip Hover	Chart tooltips display correct kWh values	Pass
ST-10	System	Error Handling	Custom 404 page displayed for invalid routes	Pass

2.5 Graphical User Interface (GUI) Layout

The EnergyGuard platform features a modern, responsive, and security-focused Graphical User Interface (GUI) designed to support both high-performance energy monitoring and secure identity management. The interface is built with a professional dark-themed design that enhances visual clarity, reduces eye strain, and gives the platform a modern technological appearance. Its organized grid-based structure allows users to view important information quickly while maintaining a clean and user-friendly experience.

The GUI is designed to combine energy analytics, system monitoring, and authentication features into one unified environment, ensuring that users can easily manage both operational data and security controls.

• Dashboard as the Central Control Hub:

The main dashboard acts as the core interface of the platform, providing users with a

centralized location to monitor energy performance, access system controls, and view critical updates after secure login.

- **Color-Coded Stat Cards:**

The dashboard includes visually distinct stat cards that display key real-time energy metrics such as energy usage, solar generation, grid load, and estimated cost. Different colors improve readability and help users quickly identify performance categories.

- **7-Day Consumption Chart:**

An interactive chart provides weekly energy consumption trends, allowing users to analyze usage patterns, compare daily performance, and support better energy decision-making.

- **Device Monitoring Grid:**

The interface includes a structured monitoring section for connected devices or infrastructure components, enabling users to track system performance and operational status efficiently.

- **Security Alert Feed:**

A dedicated alert panel displays important security notifications, suspicious activity warnings, or system alerts in real time. This ensures that users remain informed about both platform security and operational issues.

- **Authentication Interface:**

The platform includes a split-panel Sign Up and Login interface that securely manages user access through encrypted authentication workflows. Clear forms, validation messages, and secure session entry improve both usability and protection.

- **Responsive Multi-Device Accessibility:**

The GUI is designed to adapt across desktops, laptops, and other screen sizes, ensuring accessibility and consistent performance on multiple devices.

- **Unified User Experience:**

By integrating dashboard analytics, device tracking, authentication, and security notifications into a single interface, the GUI ensures that critical system data is accessible at a glance without overwhelming the user.

Overall GUI Importance:

The GUI layout of EnergyGuard is designed not only for visual appeal but also for functionality, clarity, and security. Its responsive dark-themed dashboard, interactive monitoring tools, and secure authentication system create a professional interface that supports efficient energy management while maintaining strong identity protection. This design enhances user experience by making complex data easier to understand, monitor, and control in a secure digital environment.

2.6 Customer testing

Customer testing was conducted to evaluate whether the Identity Security Framework for Energy is user-friendly, accessible, and easy to operate while maintaining strong security standards. Since the platform is designed for both practical and educational use, testing focused on user experience, interface clarity, ease of navigation, and accessibility for both normal users and administrators. The goal was to ensure that even beginner-level users could interact with the system confidently without technical difficulty.

• Signup Simplicity:

Testing was conducted to verify that new users can easily create accounts through a simple and understandable registration process. Users were able to enter details, follow clear form instructions, and complete account creation without confusion.

• Password Error Visibility:

The system was tested to ensure that users clearly understand password mistakes, invalid credentials, or incomplete form entries through proper validation messages and visible error notifications. This improved user confidence and reduced confusion during login attempts.

• Login Speed and Ease:

Customer testing confirmed that users can log in quickly and securely through a clear login interface without unnecessary delays or complicated steps. This improved accessibility and overall satisfaction.

• Dashboard Clarity:

The dashboard was tested for visual clarity, simplicity, and ease of understanding. Users

confirmed that important information, navigation controls, and system features were clearly presented, making the platform beginner-friendly.

- **Admin Usability:**

Administrative users tested the platform's management controls to ensure that account moderation, user deletion, and administrative tasks could be performed easily without technical complexity.

- **Navigation and Accessibility:**

Testing also confirmed that users could move smoothly between signup, login, dashboard, and logout functions with minimal confusion, supporting a consistent and accessible experience.

2.7 Evaluation

The Identity Security Framework for Energy successfully achieved its primary educational and functional objectives by developing a secure, lightweight, and practical authentication system for protected energy platform access. The project effectively demonstrates how essential cybersecurity principles such as user authentication, password protection, session management, route security, and administrative moderation can be implemented within a real-world web application environment.

By combining Flask, SQLite, SQLAlchemy, and Flask-Login, the system provides a clear example of how multiple security components work together to protect user identities and restrict unauthorized access. As an educational model, the project not only fulfills technical requirements but also serves as a practical demonstration of secure web application development.

Strengths

The project includes several important strengths that make it effective, secure, and suitable for academic or prototype deployment:

- **Secure Password Storage:**

User passwords are securely hashed using Werkzeug before being stored in the database, ensuring that credentials are protected from direct exposure. This significantly improves user security and demonstrates proper password protection practices.

- **Session Security:**

Flask-Login provides reliable session management by maintaining authenticated user sessions securely and restricting unauthorized access to protected routes such as dashboards and admin controls. This ensures secure login persistence and proper logout handling.

- **Lightweight Deployment:**

The system is designed to run efficiently on standard desktops or laptops without requiring advanced hardware, cloud servers, or enterprise infrastructure. This makes it cost-effective, accessible, and highly suitable for student projects or local deployment.

- **SQL Injection Defense:**

By using SQLAlchemy ORM instead of direct SQL queries, the project improves database security and reduces risks such as SQL injection attacks. This also simplifies database management and improves code maintainability.

- **Admin Moderation and Role-Based Access:**

The inclusion of admin-only controls allows administrators to manage users, remove unauthorized accounts, and maintain overall system security. This demonstrates practical implementation of authorization and governance features.

- **Educational Value:**

The project clearly explains how authentication systems function in practice, making it a strong learning tool for understanding identity management, secure coding, and web security fundamentals.

Limitations

While the project performs effectively as a prototype, certain limitations remain that affect scalability and advanced security capability:

- **SQLite is Local-Only:**

SQLite is suitable for lightweight academic projects and local deployment, but it may not perform efficiently for large-scale enterprise applications with multiple concurrent users or cloud-based infrastructure.

- **No OTP or Two-Factor Authentication (2FA):**

The system currently relies on standard password-based authentication only. Advanced security measures such as OTP verification, email confirmation, or multi-factor authentication are not included, which limits higher-level security readiness.

- **Mock Dashboard Data:**

The energy dashboard currently uses simulated or sample data rather than real-time energy monitoring from actual devices or APIs. This limits practical deployment in live energy management systems.

- **Limited Scalability:**

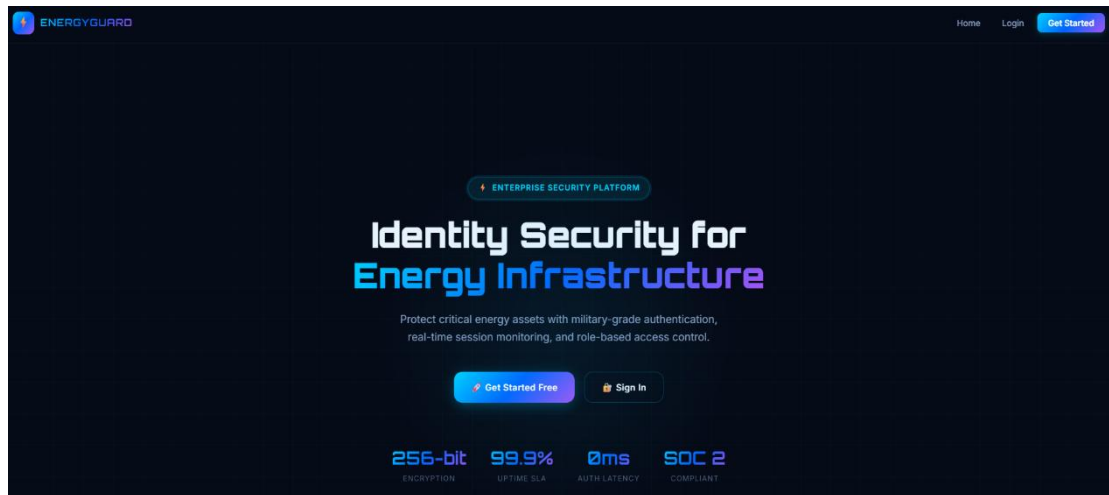
The current architecture is designed primarily for educational and prototype purposes. Large-scale deployment would require stronger databases, advanced hosting, and expanded infrastructure.

- **Basic Security Scope:**

Although core authentication is secure, features such as account recovery, password reset, login attempt monitoring, and intrusion detection are not yet implemented..

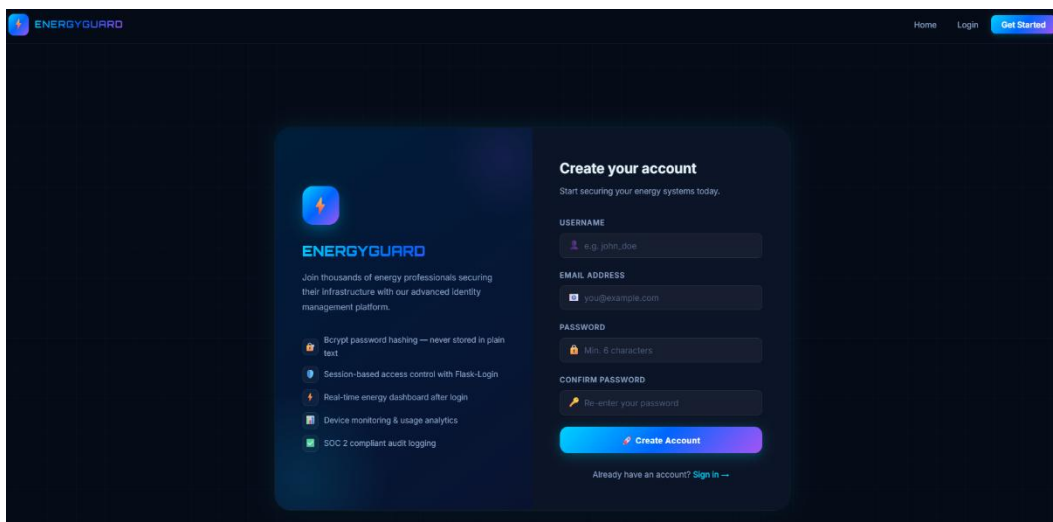
3. SNAPSHOTS OF THE PROJECT

1.Home page :



The Home Page serves as the primary landing interface for the EnergyGuard platform. It features a modern, high-impact hero section titled "Identity Security for Energy Infrastructure." The design emphasizes the platform's professional nature with a dark-themed "Glassmorphism" aesthetic. It provides users with immediate navigation options to "Sign In" or "Get Started Free," while prominently displaying key system metrics such as 256-bit encryption standards and SOC 2 compliance readiness to build user trust.

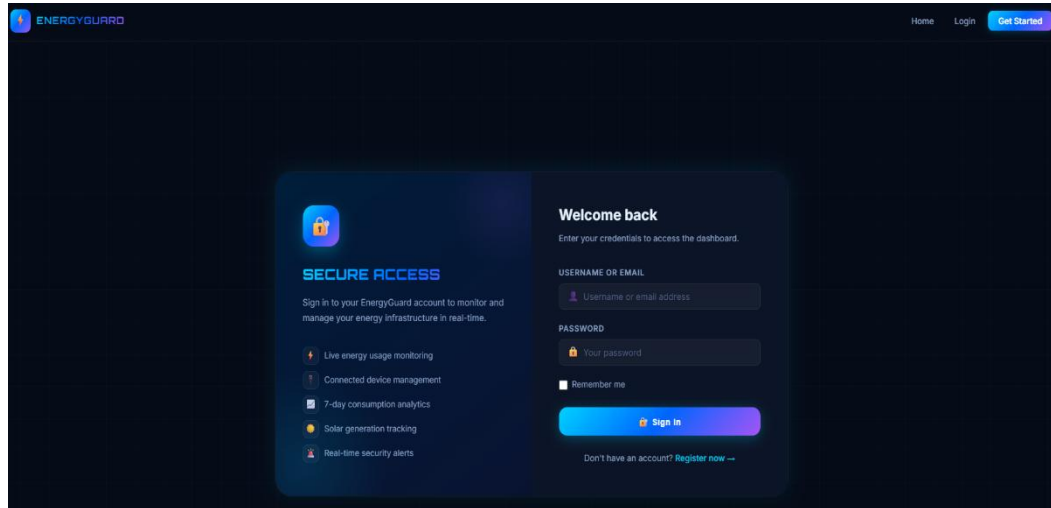
2.Register :



The Registration Page is designed to facilitate secure user onboarding. It features a split-layout interface where the left panel highlights the core security features of the platform (e.g., Bcrypt password hashing and RBAC). The right panel contains a streamlined "Create

your account" form with real-time validation for usernames, email addresses, and password matching. This ensures that only valid data enters the system while educating the user on the security measures protecting their new account.

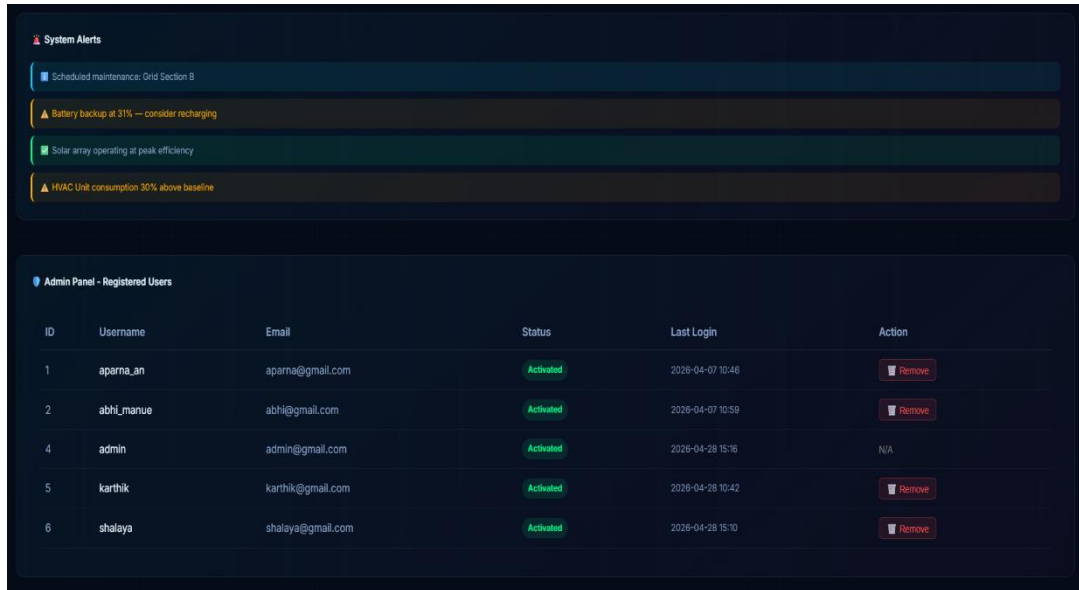
3.Login :



The Login Page provides a secure gateway for authorized users. Similar to the registration page, it maintains visual consistency and highlights "Secure Access" benefits. The form supports multi-identifier login (Username or Email) and includes a "Remember Me" feature utilizing secure, encrypted session cookies. This interface is the first line of defense, ensuring that only authenticated individuals can access sensitive energy infrastructure data.

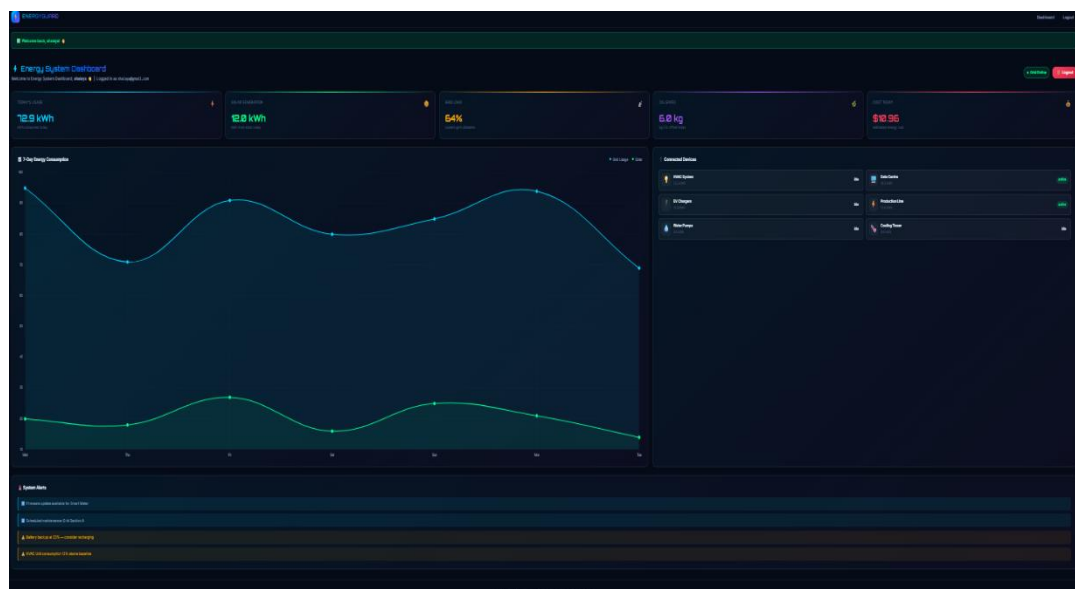
4.Admin Panel :





The Admin Panel is a specialized view restricted to users with administrative privileges. As shown in the snapshots, it integrates high-level system alerts at the top (e.g., maintenance schedules and battery backup warnings) followed by a comprehensive "Registered Users" management table. Administrators can monitor user status, track "Last Login" timestamps for auditing, and perform moderation actions such as removing unauthorized or inactive accounts to maintain system integrity.

5. Dashboard :



The Dashboard is the central command center for real-time energy monitoring. It displays critical Key Performance Indicators (KPIs) such as Today's Usage, Solar Generation, Grid

Load, and Estimated Cost. The interface features dynamic, interactive line charts powered by Chart.js that visualize energy consumption versus solar production trends. Additionally, a "Connected Devices" panel provides granular visibility into individual component status (e.g., HVAC System, EV Chargers), allowing users to make data-driven decisions to optimize energy efficiency.

4. CONCLUSION

The Identity Security Framework for Energy successfully demonstrates how secure authentication and authorization mechanisms can be used to protect modern energy platforms from unauthorized access and misuse. By integrating Flask as the web framework, SQLite for lightweight database management, password hashing for secure credential storage, and session management for controlled user access, the project creates a reliable and protected digital environment. In this system, only verified users are allowed to access sensitive energy dashboards, ensuring that critical data and platform controls remain secure.

This project clearly emphasizes essential cybersecurity concepts that are necessary for energy sector applications. Features such as secure user registration, protected login systems, encrypted password storage, session-based user authentication, and admin-level access management work together to strengthen system security. These elements help prevent common security threats such as unauthorized login, password theft, and misuse of administrative privileges. Through this implementation, the project provides practical insight into how identity security forms the backbone of safe digital operations in energy management systems.

Another major strength of the project is its lightweight and cost-effective design. Since it uses simple technologies like Flask and SQLite, the framework is easy to develop, deploy, and maintain without requiring expensive infrastructure. This makes it especially suitable for students, researchers, and small-scale organizations who want to understand or implement identity security in a practical way. Despite its simplicity, the project successfully delivers real-world functionality and demonstrates how even basic security layers can significantly improve system protection.

Overall, the Identity Security Framework for Energy proves that identity security is a fundamental first step in safeguarding modern energy systems. It establishes a strong security foundation upon which more advanced technologies can be built in the future. Potential enhancements may include multi-factor authentication, biometric verification, cloud deployment, AI-based threat detection, and real-time energy monitoring integration. By combining strong security practices with scalability, this project not only meets current

educational and functional goals but also opens the path toward more advanced, intelligent, and secure energy management solutions.

5. FURTHER DEVELOPMENT

The Identity Security Framework for Energy can be further improved by integrating advanced security mechanisms, modern infrastructure, and practical real-world technologies to make the system more secure, scalable, and suitable for larger deployments. While the current framework effectively demonstrates secure authentication and authorization, future enhancements can significantly expand both its cybersecurity strength and operational capabilities. These developments can help transform the project from a student-level prototype into a more powerful enterprise-ready energy security platform.

- **OTP / Multi-Factor Authentication:** Adding One-Time Passwords (OTP) or Multi-Factor Authentication (MFA) can provide an additional security layer during the login process. In addition to entering a password, users would need to verify their identity through email codes, SMS verification, or authentication apps. This greatly reduces the risk of unauthorized access even if passwords are stolen, making the system far more secure for sensitive energy platform access.
- **Cloud Database Integration:** Replacing SQLite with cloud-based database systems such as MySQL, PostgreSQL, or cloud-hosted database services can improve scalability, storage capacity, and remote accessibility. Cloud integration would allow the system to support larger user bases, real-time synchronization, and secure backup, making it more suitable for industrial or commercial energy deployments.
- **Real-Time Smart Meter APIs:** Integrating live smart meter or IoT-based energy APIs can convert the dashboard from simulated data into a real-time monitoring platform. This would allow users to track actual electricity consumption, energy efficiency, and system anomalies instantly, making the framework more practical for smart buildings, factories, and energy management systems.
- **Role-Based Access Control (RBAC):** Advanced role management can assign different permission levels to users, managers, technicians, and administrators. This ensures that each user can only access features relevant to their role, reducing misuse, improving data privacy, and strengthening organizational security management.

- **AI-Based Intrusion Detection:** Artificial Intelligence can be integrated to monitor login patterns, user behavior, and suspicious activities. AI-powered systems can automatically detect unusual access attempts, brute-force attacks, or abnormal behavior and provide alerts or preventive actions. This would enhance proactive cybersecurity defense for the energy platform.
- **Password Reset via Email:** A secure email-based password recovery system can improve user convenience while maintaining security. Users who forget their passwords can safely reset them through verified email links or tokens, improving accessibility without compromising account protection.
- **Audit Logs:** Implementing detailed audit logs can record user activities such as logins, failed login attempts, dashboard access, and administrative changes. These logs help monitor security events, improve accountability, support troubleshooting, and provide valuable evidence in case of suspicious activity or system breaches.

Overall, these future developments can greatly enhance the Identity Security Framework for Energy by making it more intelligent, secure, and adaptable to modern cybersecurity challenges. By incorporating stronger authentication, cloud technologies, AI-driven security, real-time energy monitoring, and comprehensive user management, the project can evolve into a highly advanced, scalable, and enterprise-level platform capable of protecting next-generation digital energy systems.

6. REFERENCES

The following references were used for developing the Identity Security Framework for Energy project. These resources helped in understanding web development, authentication, database management, frontend design, and security implementation.

- **Flask Documentation:** Used for web application development and backend routing.

<https://flask.palletsprojects.com/>

- **Flask-Login Documentation:** Used for user authentication, session management, and access control.

<https://flask-login.readthedocs.io/>

- **Flask-SQLAlchemy Documentation:** Used for database integration and ORM functionality.

<https://flask-sqlalchemy.palletsprojects.com/>

- **SQLite Documentation:** Used for local database storage and data management.

<https://sqlite.org/>

- **Werkzeug Security Documentation:** Used for password hashing and credential protection.

<https://werkzeug.palletsprojects.com/>

- **Python Documentation:** Used for core programming and application logic.

<https://docs.python.org/3/>

- **MDN Web Docs (HTML/CSS):** Used for frontend design and webpage structure.

<https://developer.mozilla.org/>

- **OWASP Security Guidelines:** Used for secure authentication and cybersecurity best practices.

<https://owasp.org/>

These references provided the technical and security foundation for building the project successfully.

7.APPENDIX

This appendix provides a detailed overview of the core database structure, application routes, security functions, system flow, and project directory structure used in the “Identity Security Framework for Energy” project. These components form the technical foundation of the system and demonstrate how authentication, user management, and secure access control are implemented in a real-world energy platform.

Project Overview

The Identity Security Framework for Energy project was designed to create a secure and user-friendly authentication system for energy management platforms. It combines backend development, frontend design, secure password protection, database management, and session handling into one integrated web application.

Sample User Database Schema

The database stores user credentials and account-related details required for authentication and access control.

User Table Structure

Field Name	Data Type	Purpose
user_id	Integer (Primary Key)	Unique identification for each user
username	String	User's chosen login name
email	String	Registered email address
password_hash	String	Securely encrypted password
last_login	DateTime	Tracks recent login activity

Schema Explanation:

- **user_id:** Prevents duplicate user identity
- **username/email:** Used for login and communication
- **password_hash:** Stores encrypted password instead of plain text
- **last_login:** Useful for security tracking and account monitoring

Database Schema Diagram

USER TABLE
user_id (Primary Key)
username
email
password_hash
last_login

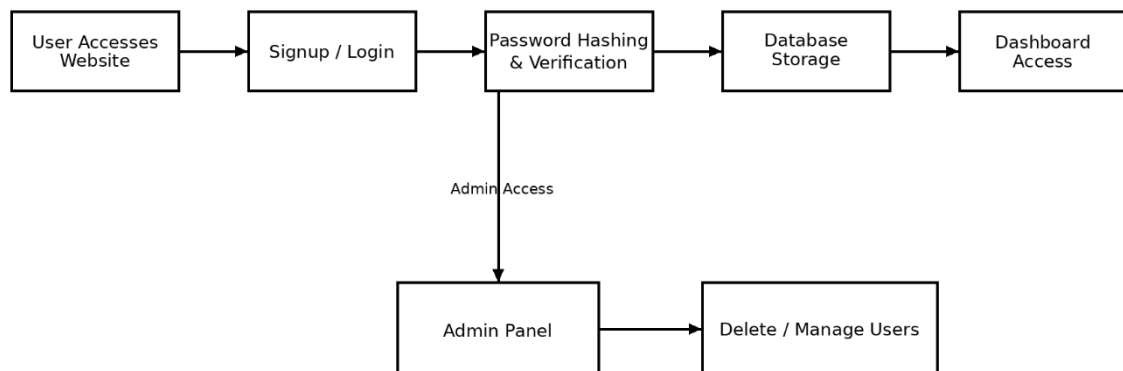
Sample Application Routes

Application routes define how users interact with the system through webpages and backend actions.

Route	Function
/signup	Registers new users
/login	Authenticates users
/logout	Ends user session
/dashboard	Secure user homepage
/admin/delete_user/<id>	Admin removes users

Route Flow:

1. User signs up
2. Password is hashed
3. User logs in
4. Session starts
5. Dashboard access granted
6. Admin can manage users

Route Flow Diagram

Normal User Flow: Registration → Authentication → Database → Dashboard

Security Functions

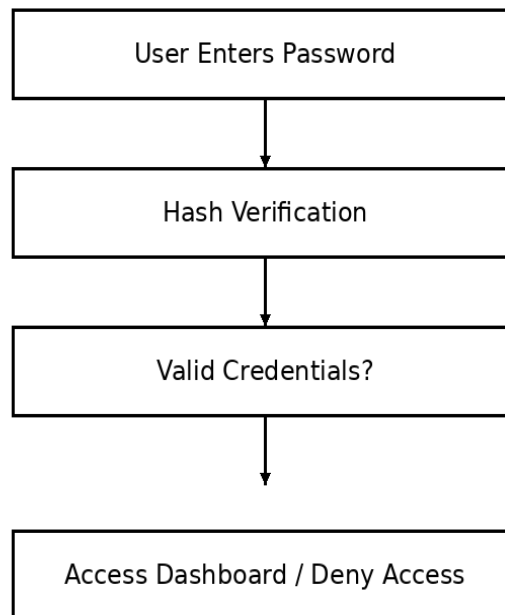
Security is a critical part of the project and protects user credentials and access permissions.

Core Security Functions:

- generate_password_hash()
 - Encrypts passwords before storing in database
 - Prevents password theft
- check_password_hash()
 - Verifies entered password during login
 - Ensures secure authentication
- login_user()
 - Creates user session

- Grants access to protected pages
- `logout_user()`
- Safely ends user session
- Prevents unauthorized reuse

Authentication Flow Diagram



Frontend Technologies Used

The frontend interface was designed for simplicity and usability.

Technologies:

- HTML – Page structure
- CSS – Styling and layout
- Bootstrap (optional) – Responsive design
- Jinja Templates – Dynamic Flask page rendering

Main Features:

- Registration Form
- Login Form
- Dashboard Interface
- Admin Control Panel

Project Directory Structure

Below is the recommended project folder structure:

```
IdentitySecurityFramework/  
├─ app.py  
├─ users.db  
├─ templates/  
│   ├─ signup.html  
│   ├─ login.html  
│   └─ dashboard.html  
└─ admin.html  
├─ static/  
│   └─ style.css  
└─ requirements.txt
```

Directory Explanation:

- **app.py:** Main Flask backend logic
- **users.db:** SQLite database
- **templates:** HTML pages
- **static:** CSS styling
- **requirements.txt:** Required libraries

System Workflow Summary

Full System Process:

1. User accesses signup/login page
2. Data is validated
3. Password is securely hashed
4. Information is stored in database
5. User logs in securely
6. Session is created

7. Dashboard access is provided
8. Admin manages users if required

Security Best Practices Applied

Implemented Practices:

- Password hashing
- Session authentication
- Protected routes
- Admin authorization
- Secure credential storage
- OWASP-inspired authentication design

Appendix Conclusion

This appendix demonstrates the practical technical implementation of the Identity Security Framework for Energy project by combining secure authentication, protected user management, structured backend routing, database design, and cybersecurity best practices. These components collectively ensure that the system is secure, efficient, scalable, and suitable for educational or real-world deployment.